

JPEG Image Compression

Multimedia Databases SS26 (Exercises)

Prof. Dr. Harald Kosch / Prof. Dr. Mario Döller

Tutor: Manuel Lehner



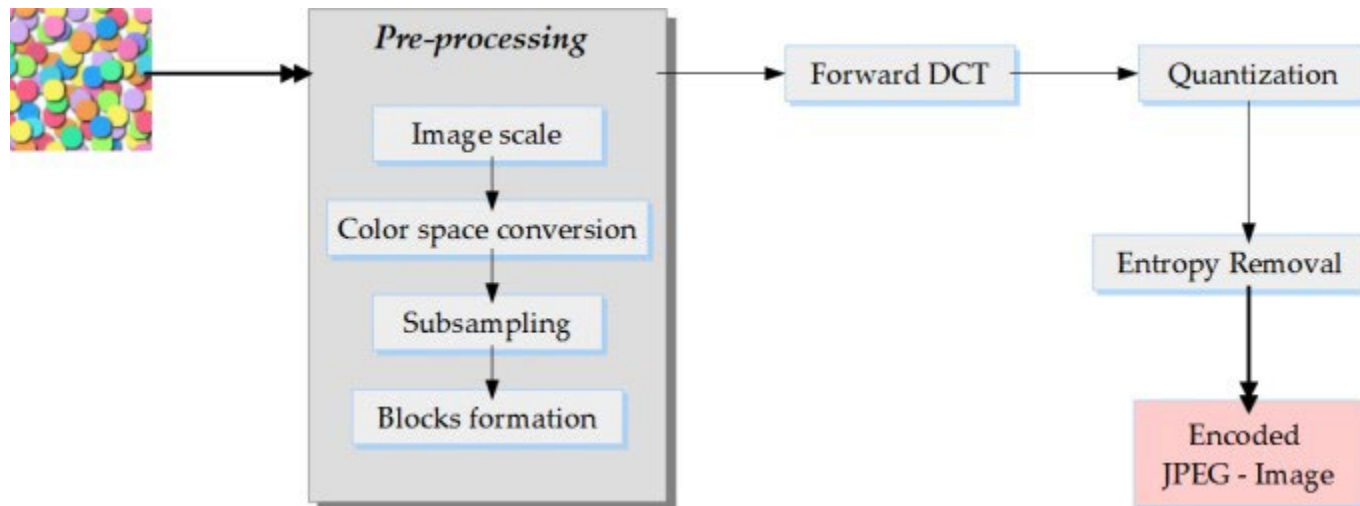
Outline

- JPEG Baseline Process
- Pre-processing
- Forward Discrete Cosinus Transformation (F-DCT)
- Quantization
- Entropy Coding



Task 1.1: JPEG compression algorithm

- Baseline Process
 - **Goal:** Lossy compression with an inverse decompression



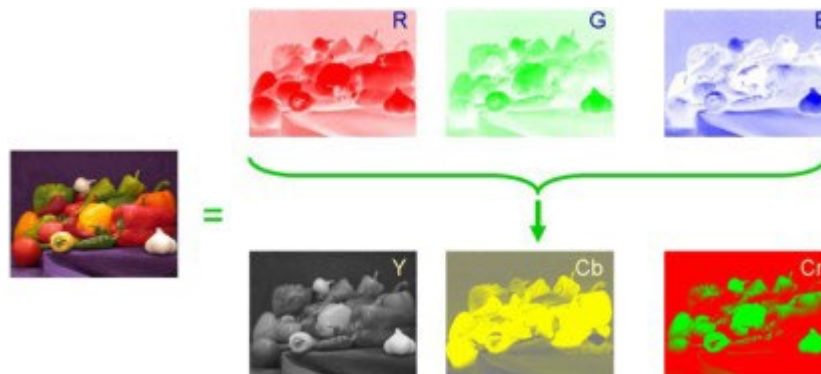
Task 1.2

- Pre-processing:
 - Image scale + Blocks formation: divide the image into blocks of equal/specific size: Lossless.
 - Color space conversion: Lossless.
 - Subsampling: remove less significant details: Lossy
- DCT: Transform the image to frequency domain: Lossless
- Quantization: Remove less significant details: Lossy
- Entropy Removal: Remove redundancy: Lossless



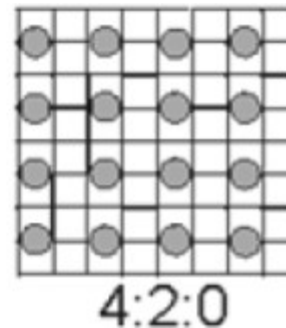
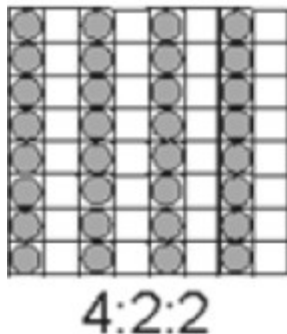
Task 2.1: Pre-processing: Color Space

- **Conversion from RGB to YUV (YC_bC_r)**
- **Separates luma from chroma**
 - Luma (luminance): Y , isolated
 - Chroma (Chrominance): two color-difference components
 - C_b blue – luma
 - C_r red – luma
- **Enable data-reduction, through the fact:**
 - Human perception is less sensitive to changes in color than light intensity



Task 2.2: Pre-processing: Subsampling

- **Data reduction**
 - Keep all luma coefficients
 - Discard parts of chroma coefficients
- **Given by a ratio: $X:a:b$**
 - X : Reference
 - a : num of C_b or C_r per odd line
 - b : num of C_b or C_r per even line
- $4:4:4 \rightarrow$ no changes



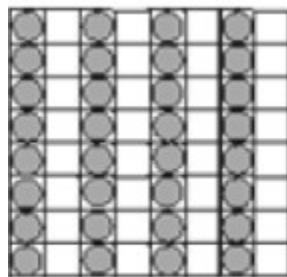
Task 2: Pre-processing: Image Scale

- **DCT can only work with blocks of size 8x8**
 - Image dimensions must be divisible by 8
- **Missing pixels are completed as follows**
 - Extra columns on the right contain same value as furthestmost column
 - Extra rows below contain same value as the last row of the image
 - Missing block on the bottom right contains same value as the last pixel of image

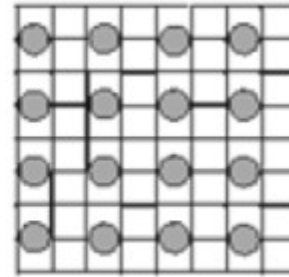


Task 2.3: Pre-processing: Blocks Formation

- Image must be divided into blocks of MCU (Min Coded Units): 8x8
 - **But: Subsampling drops chroma coefficients**
 - So there might not be enough elements
 - 4:4:4 makes no changes $\rightarrow$ MCU = 8 x 8
 - 4:2:0 contains 50% in each dimension $\rightarrow$ MCU = 16 x 16
- $\rightarrow$ Image width/height must be divisible by 16 (or scaled)



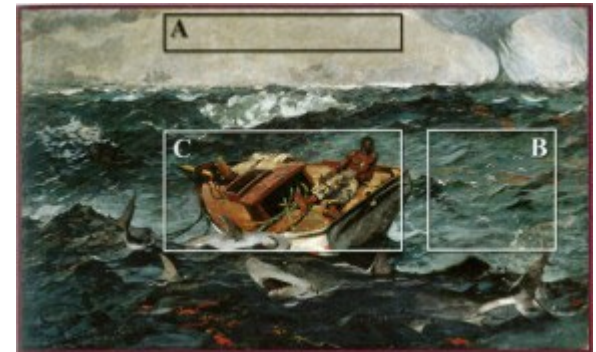
4:2:2



4:2:0

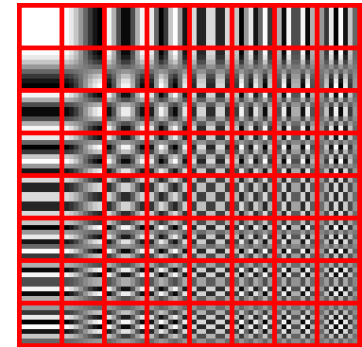
Task 3.1: DCT: Discrete Cosinus Transform.

- **DCT translates image information of space domain (visible form of representation) into frequency domain**
 - Applied on 8x8 blocks
 - MUST be reversible
 - No data reduction during transformation, but basis for further processing
- **Frequency Domain:**
 - Frequency is the rate at which pixel values change over spatial distance
- Color Variance $\rightarrow$ Frequency
 - A: Low, B: Moderate, C: High
- Transformation helps separating less important from more important information.
 - Human eye is most sensitive to low frequencies than high frequencies.



Task 3.2 and 3.3: F-DCT: Coefficients

- **Image is divided into 8x8 blocks**
 - Each is considered as discrete signal with 64 coefficients
- **Two types of coefficients**
 - DC:DCT(0,0) Gives the basic hue of the image
 - AC:The other 63 coefficients
 - Represent color change across the block
- **Further parts (higher frequencies) are the more useful parts for compression.**



Task 4.1: Quantization

- **Quantization is performed by:**
 - Divide each element in the transformed matrix $F(u,v)$ by the corresponding element in the quantization matrix
 - Round to nearest integer value
- **Quantization matrix is influential**
 - Not standardized, but recommendations are provided
 - Quantization tables for chroma and luma based on subjective experiments involving human visual system

162.3	40.6	20.0	72.3	30.3	12.5	-19.7	-11.5
30.5	108.4	10.5	32.3	27.7	-15.5	18.4	-2.0
-94.1	-60.1	12.3	-43.4	-31.3	6.1	-3.3	7.1
-38.6	-83.4	-5.4	-22.2	-13.5	15.5	-1.3	3.5
-31.3	17.9	-5.5	-12.4	14.3	-6.0	11.5	-6.0
-0.9	-11.8	12.8	0.2	28.1	12.6	8.4	2.9
4.6	-2.4	12.2	6.6	-18.7	-12.8	7.7	12.0
-10.0	11.2	7.8	-16.3	21.5	0.0	5.9	10.7

DCT matrix $F(u,v)$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Quantization matrix
 $Q(u,v)$ 

10	4	2	5	1	0	0	0
3	9	1	2	1	0	0	0
-7	-5	1	-2	-1	0	0	0
-3	-5	0	-1	0	0	0	0
-2	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

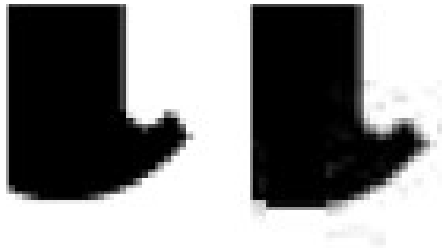
Task 4.2: Quantization

- **Advantages**

- Compression: Save bits by restricting value range
 - Reduce frequency components that are negligible for the image information

- **Disadvantages**

- Less possible values → worst quality
 - Lossy operation
- Quantization error (noise) occurs → edge artifacts



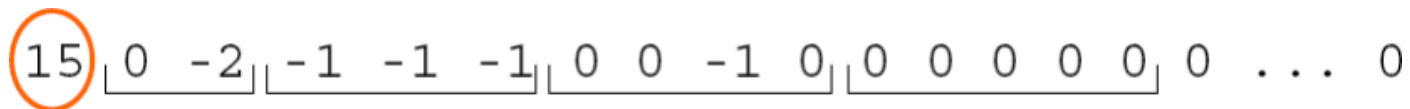
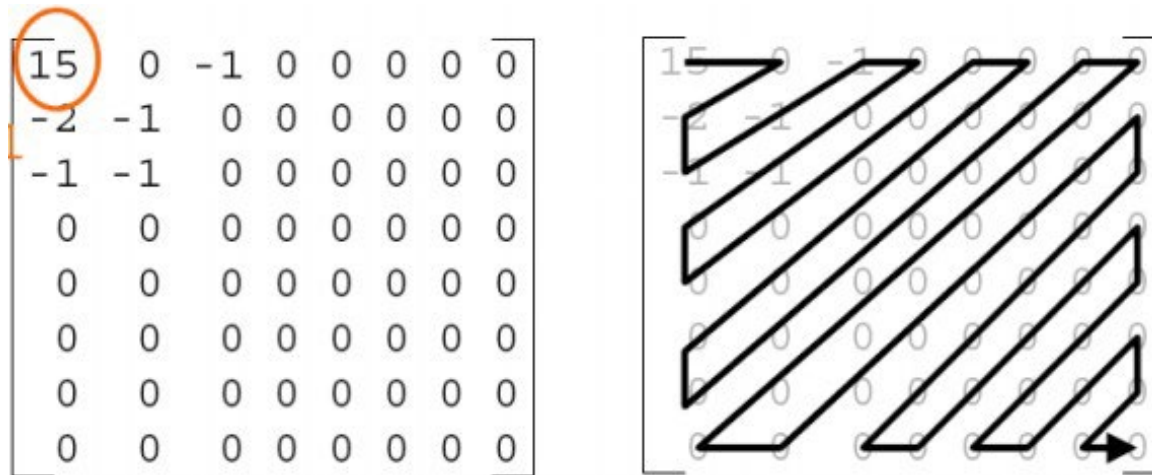
Task 5.1: Entropy Coding

- **Purpose of entropy coding**
 - Last step of JPEG compression
 - Reduce redundancy in quantized coefficients
 - Lossless
- **Main contribution of RLE to data compression**
 - Run-length encoding helps reducing data by compressing series of identical bits
 - A block must be handled first by a zig-zag scan



Task 5.1: Entropy Coding: Zig-Zag

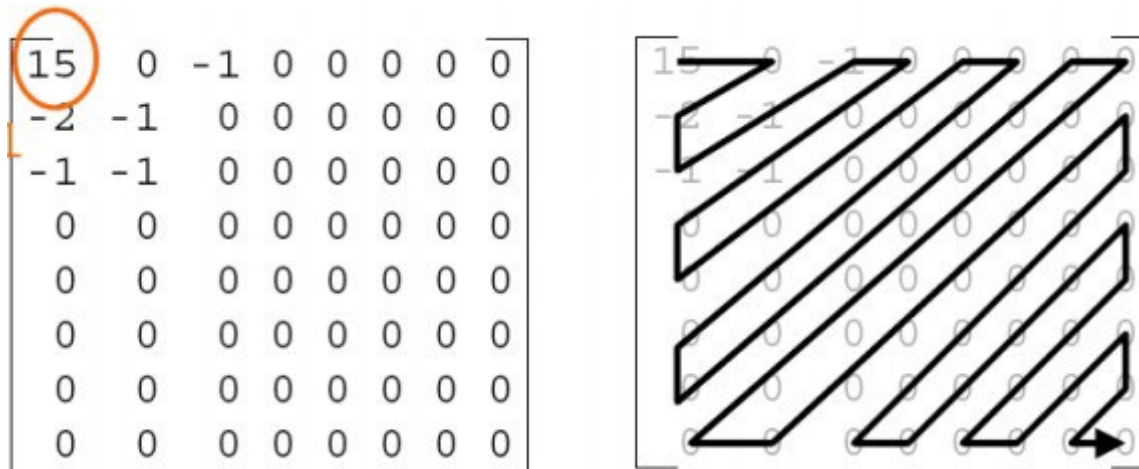
- **The Zig-Zag scan maps an 8x8 matrix to a 1x64 vector**
 - Group low frequency coefficients at vector's front and high frequency coefficients at bottom.



Task 5.2: Entropy Coding: RLE

- **DC- and AC-coefficients are entropy coded differently**
- **RLE (and zig-zag) is applied only for AC coefficients**
 - Due quantization step, the 1x64 vectors have a lot of zeros in them
 - Keeps *skip* and *value*, where *skip* is the number of zeros and *value* is the next non-zero component.

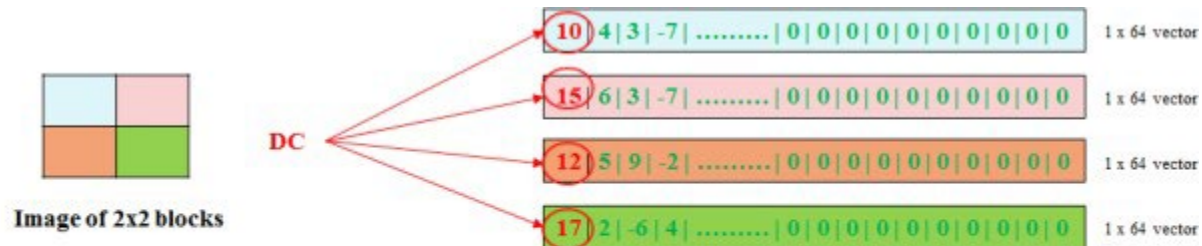
0 -2 -1 -1 -1 0 0 -1 0 0 0 0 0... $\rightarrow$ **1**, -2, **0**, -1, **0**, -1, **0**, -1, **2**, -1 **56**



Task 5.3: Entropy Coding: Difference Coding

- **For DC coefficients:**

- Difference between the coefficients of neighboring blocks is used for getting higher compression rates
 - Neighboring DC coefficients are similar
 - Differences between them are very small


$$\Rightarrow 10, 5, -3, 5, ..$$

Task 6: LZW Encoding

- Lempel–Ziv–Welch (LZW) is a universal, lossless data compression algorithm created by Abraham Lempel, Jacob Ziv, and Terry Welch.
- Published by Welch in 1984 as an improved implementation of the LZ78 algorithm published by Lempel and Ziv.
- Used in the GIF image format.
- A large English text file can typically be compressed via LZW to about half its original size.



Task 6: LZW Encoding

1. The **Prefix** is empty and the dictionary is initialized with all characters
2. Read the next character **z** from the input stream
3. Is “**Prefix + z**” in the dictionary?

Yes: **Prefix** = **Prefix** + **z**

No:

Output Code for Prefix

put **Prefix + z** in the dictionary

Prefix = **z**

4. Has the end of the input stream been reached?

No: go to step 2

Yes: If the **Prefix** is not empty, output the corresponding code



Z	P	P+z	P+z in Dictionary?			
			Yes	No		
			New P	Output	Dictionary	New P
					1:T,2:A,3:R	
T		T	T			
A	T	TA		1	4:TA	A
T	A	AT		2	5: AT	T
T	T	TT		1	6:TT	T
A	T	TA	TA			
R	TA	TAR		4	7:TAR	R
R	R	RR		3	8:RR	R
A	R	RA		3	9:RA	A
T	A	AT	AT			
T	AT	ATT		5	10:ATT	T
A	T	TA	TA			
T	TA	TAT		4	11:TAT	T
-	T	T		1		

Task 6: LZW Decoding

Dictionary initialised with all occurring characters

1. **Code** = first code from the input stream (always one character)
2. Output the entry for code
3. Store **code** in **OldCode**
4. **Code** = next code in the input stream
5. **Prefix** = decoding of **OldCode**
6. **Code** in dictionary?

Yes:

- Output the decoding of **Code**
- **Character** = first character of decoding of **Code**
- Enter **prefix+character** in dictionary

No:

- **Character** = first character of decoding of **OldCode**
- Enter **prefix+character** in dictionary AND output it As long as characters still exist, go to 4.



Task 6: LZW Decoding

C	oldCode	Dict ?	Prefix =Decoding(oldCode)	Character(Di ct/!Dict) =Decoding (C/oC)[0]	Dictionary 1:T,2:A,3: R	Output(Dict/!Dict) Decoding(C)/P refix+character
1						T
2	1	y	T	A	4:TA	A
1	2	y	A	T	5:AT	T
4	1	y	T	T	6:TT	TA
3	4	y	TA	R	7:TAR	R
3	3	y	R	R	8:RR	R
5	3	y	R	A	9:RA	AT
4	5	y	AT	T	10:ATT	TA
1	4	y	TA	T	11:TAT	T



Task 7: Huffman-Code

- **Generate the Huffman Code for the following set of characters**

A	R	Y	O	S	T	X	U
30.1%	17.5%	21.5%	14.9%	9.3%	2.2%	2.3%	2.2%

- Create a leaf node for each symbol and add it to the priority queue.
- While there is more than one node in the queue:
 - Remove the two nodes of highest priority (lowest probability) from the queue
 - Create a new internal node with these two nodes as children and with probability equal to the sum of the two nodes' probabilities.
 - Add the new node to the queue.
- The remaining node is the root node and the tree is complete.



Huffman Coding



A 30.1%



R 17.5%



T 2.2%



O 14.9%



X 2.3%



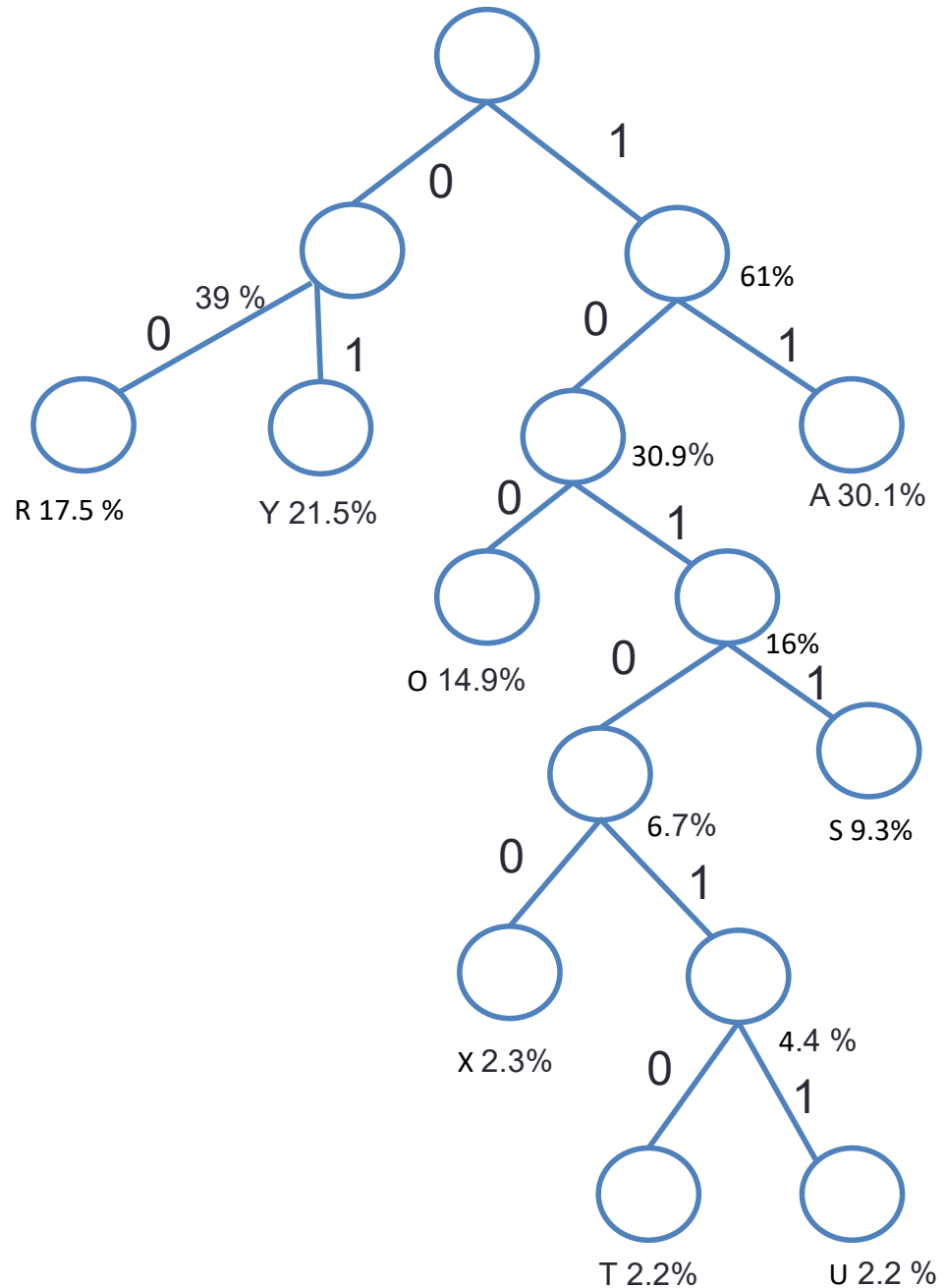
Y 21.5%



U 2.2 %



S 9.3%



Task 7: Huffman Coding

A	R	Y	O	S	T	X	U
30.1%	17.5%	21.5%	14.9%	9.3%	2.2%	2.3%	2.2%

A	11
Y	01
R	00
O	100
S	1011
X	10100
T	101010
U	101011